FACULDADE FACINT

ENGENHARIA DE SOFTWARE

Estruturas Lineares

Organização e Manipulação de Estruturas de Dados

Arrays · Listas Encadeadas · Pilhas · Filas · LIFO · FIFO

🎯 Objetivos de aprendizagem

🧱

Compreender

o conceito de estrutura linear e suas características

📦

Dominar

arrays: índices, acesso, inserção, remoção e custos

🔗

Entender

listas encadeadas: nós, referências e operações

📚

Aplicar

o princípio LIFO em pilhas e o princípio FIFO em filas

💻

Implementar

estruturas lineares em JavaScript

⚖️

Comparar

estruturas e escolher a mais adequada para cada situação

🧱 O que são estruturas lineares?

Uma **estrutura linear** organiza os elementos em uma **sequência**, onde cada elemento possui:

- Um **antecessor** (exceto o primeiro)
- Um **sucessor** (exceto o último)

Os elementos são acessados de forma **ordenada**, um após o outro.

Analogia

Uma fila de banco: cada pessoa está atrás de outra, em uma única direção. Não é possível "pular" para o meio sem passar pelas anteriores.

Estrutura linear = elementos organizados em **sequência única**, com **ordem definida**.

🔄 Lineares × Não Lineares

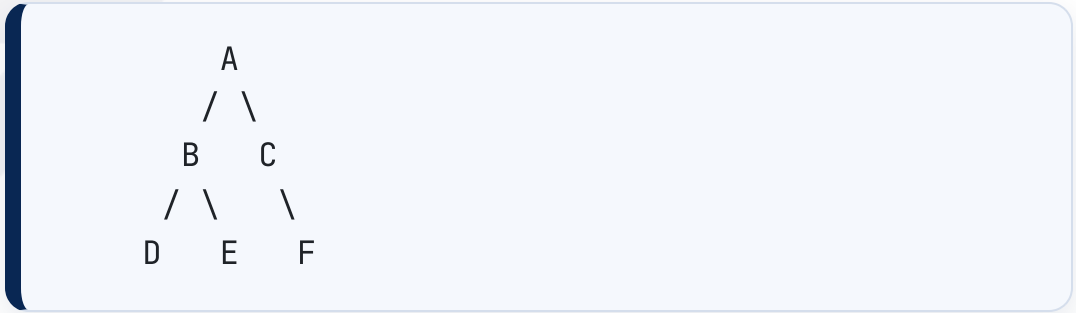
📏 Estruturas Lineares

- Elementos em **sequência**
- Cada elemento tem **um** antecessor e **um** sucessor
- Exemplos: **array, lista, pilha, fila**



🌳 Estruturas Não Lineares

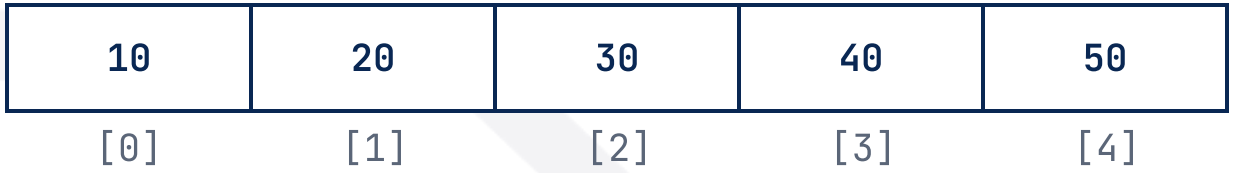
- Elementos em **hierarquia** ou **rede**
- Um elemento pode ter **vários** sucessores
- Exemplos: **árvore, grafo**



💡 Nesta aula focamos nas **estruturas lineares**. Árvores e grafos serão abordados nas aulas seguintes.

📦 Arrays e Vetores

Um **array** é uma estrutura linear que armazena elementos em **posições contíguas** de memória.



- Tamanho **fixo** (em linguagens como C) ou **dinâmico** (em JavaScript)
- Acesso **direto** por índice
- Índice começa em **0**
- Elementos do **mesmo tipo** (em linguagens estáticas)

A grande vantagem do array: acesso O(1) a qualquer elemento pelo índice.

📍 Índices e acesso aos elementos

```
const frutas = ["Maçã", "Banana", "Laranja", "Uva", "Manga"];

// Acesso direto por índice — O(1)
console.log(frutas[0]);      // "Maçã"
console.log(frutas[3]);      // "Uva"
console.log(frutas.length);  // 5

// Primeiro e último elemento
const primeiro = frutas[0];
const ultimo  = frutas[frutas.length - 1];  // "Manga"

// Tentativa de acessar índice inexistente
console.log(frutas[10]);     // undefined (não gera erro)
```

📌 **Conceito:** o acesso por índice é **instantâneo** — o computador calcula o endereço diretamente: `endereço base + índice × tamanho`.

🛠️ Criação e manipulação de arrays

```
const notas = [7.5, 8.0, 6.0, 9.5];

// push() e pop() — FINAL → O(1)
notas.push(10.0);              // [7.5, 8.0, 6.0, 9.5, 10.0]
notas.pop();                   // remove 10.0

// unshift() e shift() — INÍCIO → O(n)
notas.unshift(5.0);            // [5.0, 7.5, 8.0, 6.0, 9.5]
notas.shift();                 // remove 5.0

// splice() — POSIÇÃO ESPECÍFICA → O(n)
notas.splice(1, 0, 7.0);       // insere 7.0 na posição 1
notas.splice(2, 1);            // remove 1 elemento na posição 2
console.log(notas);            // [7.5, 7.0, 6.0, 9.5]
```

⚠️ `push` e `pop` = rápidos **O(1)**. `unshift`, `shift` e `splice` = deslocam elementos → **O(n)**.

🔍 Busca e percurso em arrays

```
const cidades = ["Curitiba", "Londrina", "Maringá", "Cascavel", "Foz"];

// indexOf() — busca linear — O(n)
const pos = cidades.indexOf("Maringá");    // 2
const existe = cidades.includes("Foz");     // true

// find() — busca com condição — O(n)
const grande = cidades.find(c ⇒ c.length > 7);

// Percurso completo — O(n)
for (const cidade of cidades) {
  console.log(cidade);
}
```

📌 Busca em array é **O(n)**: no pior caso, é preciso visitar **todos** os elementos.

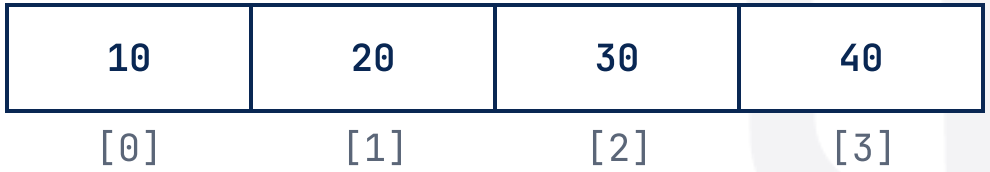
⚡ Custos das operações em arrays

Operação	Método JS	Big O	Observação
Acessar por índice	`arr[i]`	**O(1)**	Cálculo direto de endereço
Inserir no final	`push()`	**O(1)**	Amortizado
Remover do final	`pop()`	**O(1)**	Sem deslocamento
Inserir no início	`unshift()`	**O(n)**	Desloca todos os elementos
Remover do início	`shift()`	**O(n)**	Desloca todos os elementos
Inserir/remover no meio	`splice()`	**O(n)**	Desloca elementos à direita
Buscar por valor	`indexOf()`	**O(n)**	Percorre até encontrar
Percurso completo	`for...of`	**O(n)**	Visita cada elemento

🔑 **Regra:** operações no **final** do array são rápidas. Operações no **início ou meio** exigem deslocamento → custo linear.

❓ O problema do array

Inserir "X" na posição 2 de um array:



⚠️ É preciso **deslocar** 30 e 40 para abrir espaço → **O(n)**

Inserir "X" na posição 2 de uma lista:



✅ Basta **religar** os ponteiros → **O(1)**

A lista encadeada nasceu para resolver o **problema do deslocamento** em arrays.

🔗 Introdução às listas encadeadas

Uma **lista encadeada** é uma estrutura linear onde cada elemento (chamado **nó**) contém:

- O **valor** armazenado
- Uma **referência** (ponteiro) para o próximo nó

Os nós **não precisam estar contíguos** na memória — podem estar espalhados, mas estão **conectados** por ponteiros.



Cada nó aponta para o **próximo**. O último aponta para `null` (∅), indicando o **fim** da lista.

💡 **Analogia:** como uma caça ao tesouro — cada pista te diz onde está a próxima. Não há caminho direto, só a sequência.

Diferente do array: a lista **não tem índice**. Para acessar um elemento, é preciso **percorrer** a partir do início.

🔗 Array × Lista — Diferença fundamental

Array — Acesso por índice



- Memória **contígua** (lado a lado)
- Acesso **direto** por índice → O(1)
- Inserir no meio → **desloca** → O(n)

Lista — Acesso por referência



- Memória **dispersa** (espalhada)
- Acesso **sequencial** → O(n)
- Inserir no meio → **religa ponteiros** → O(1)*

Array é como uma **prateleira numerada**. Lista é como uma **corrente**: cada elo aponta para o próximo.

🧩 Anatomia de um nó

Representação visual



- **head**: referência ao primeiro nó (porta de entrada)
- Cada nó guarda: **valor** + **ponteiro** (próximo)
- Último nó aponta para **null** (fim da lista)

💡 Cada nó é uma **caixinha** com dois campos: o dado e o endereço da próxima caixinha.

Implementação em JavaScript

```javascript
// Definição de um nó
class No {
  constructor(valor) {
    this.valor = valor;
    this.proximo = null;
  }
}

// Criando os nós
const a = new No("A");
const b = new No("B");
const c = new No("C");

// Conectando os nós
a.proximo = b;
b.proximo = c;
// c.proximo já é null

// A lista começa em 'a' (head)
```

🏗️ Construindo uma lista passo a passo

```
class No {
  constructor(valor) {
    this.valor = valor;      // O dado armazenado
    this.proximo = null;     // Ponteiro para o próximo nó
  }
}

// Passo 1: Criar nós isolados
const a = new No("Ana");      // {valor: "Ana", proximo: null}
const b = new No("Bruno");    // {valor: "Bruno", proximo: null}
const c = new No("Carlos");   // {valor: "Carlos", proximo: null}

// Passo 2: Conectar os nós
a.proximo = b;    // Ana → Bruno
b.proximo = c;    // Bruno → Carlos
// c.proximo continua null (fim da lista)

// Passo 3: Definir o head
const head = a;   // A lista começa em Ana
```

📌 Antes de conectar: 3 nós **isolados**. Depois: uma **cadeia** — `head → Ana → Bruno → Carlos → null`

+ Inserção em lista encadeada

Inserir no início — O(1)

```
const novo = new No("Diana");
novo.proximo = head;   // Diana aponta para Ana
head = novo;           // head agora é Diana
```

Antes: `head → [Ana] → [Bruno] → [Carlos] → ∅`

Depois: `head → [Diana] → [Ana] → [Bruno] → [Carlos] → ∅`

Inserir no meio — O(1)*

```
// Inserir "Eva" depois de "Ana"
const anterior = buscarNo("Ana"); // O(n) para encontrar
const novo = new No("Eva");

novo.proximo = anterior.proximo;  // Eva aponta para Bruno
anterior.proximo = novo;          // Ana aponta para Eva
```

Antes: `… → [Ana] → [Bruno] → …`

Depois: `… → [Ana] → [Eva] → [Bruno] → …`

> ⚠️ A inserção em si é **O(1)**, mas **encontrar** a posição é **O(n)**.

— Remoção em lista encadeada

Remover do início — O(1)

```
head = head.proximo; // Ana sai, Bruno vira head
```

Antes: `head → [Ana] → [Bruno] → [Carlos] → ∅`

Depois: `head → [Bruno] → [Carlos] → ∅`

Remover do meio — O(n)

```
// Encontrar o nó anterior ao que será removido
let atual = head;
while (atual.proximo.valor !== "Bruno") {
  atual = atual.proximo;
}
// Pular o nó "Bruno"
atual.proximo = atual.proximo.proximo;
```

Antes: `… → [Ana] → [Bruno] → [Carlos] → …`

Depois: `… → [Ana] → [Carlos] → …`

Remoção em lista: **reajustar ponteiros**, sem deslocar elementos. Mas é preciso **percorrer** até o nó anterior.

🔁 Percurso de uma lista encadeada

```
// Função para percorrer e exibir todos os elementos
function percorrerLista(head) {
  let atual = head;            // Começa pelo primeiro nó
  let posicao = 0;

  while (atual !== null) {     // Enquanto não chegar ao fim
    console.log(`${posicao}: ${atual.valor}`);
    atual = atual.proximo;     // Avança para o próximo nó
    posicao++;
  }
}

// Com a lista: head → Ana → Bruno → Carlos → null
percorrerLista(head);
// Saída:
// 0: Ana
// 1: Bruno
// 2: Carlos
```

📌 **Percurso** é **O(n)**: é preciso visitar cada nó, um a um. Não existe acesso direto como no array.

🎯 Quando usar uma lista encadeada?

FACULDADE FACINT

✅

Use lista quando:

Inserções e remoções **frequentes** no início ou meio. O tamanho dos dados **muda muito**. Não precisa acessar por **índice**.

❌

Use array quando:

Precisa de acesso **direto** por índice. Operações no **final** (push/pop). Tamanho relativamente **estável**.

🌍 Onde listas encadeadas são usadas?

🎵

Playlists

Músicas reordenadas, adicionadas e removidas dinamicamente

↩️

Undo/Redo

Cada ação aponta para a anterior → navegar pelo histórico

🗂️

Sistemas de arquivos

Pastas e arquivos encadeados em disco

⚙️

Filas e pilhas

Implementação interna de filas e pilhas eficientes

💡 **Regra:** se você precisa **inserir/remover no meio** com frequência, a lista vence o array.

📊 Pilhas — Stack

Uma **pilha** é uma estrutura linear que funciona como uma **pilha de pratos**.

O último a entrar é o primeiro a sair — **LIFO** (*Last In, First Out*).



Operações principais

Operação	Descrição
`push()`	Empilha (insere no **topo**)
`pop()`	Desempilha (remove do **topo**)
`peek()` *	Consulta o **topo** sem remover

💡 `peek()` é um conceito do TAD Pilha — não existe nativamente no JS. Na prática: `pilha[pilha.length - 1]`.

LIFO: o **último** a entrar é o **primeiro** a sair. Só se acessa o **topo**.

⚙ Operações de uma pilha

```
const pilha = [];

// Push — empilha no topo
pilha.push("Maçã");     // [Maçã]
pilha.push("Banana");   // [Maçã, Banana]
pilha.push("Manga");    // [Maçã, Banana, Manga]

// Pop — desempilha do topo
const topo = pilha.pop(); // "Manga"

// Peek — consulta o topo sem remover
// (peek() é do conceito de Pilha; em JS usamos o último índice)
console.log(pilha[pilha.length - 1]); // "Banana"
```

Estado da pilha

Após 3× push:



Após pop():



💻 Pilha em JavaScript — exemplo comentado

```
// Histórico de ações de um editor (Ctrl+Z)
const historico = [];

historico.push("Digitou 'Olá'");
historico.push("Negritou texto");
historico.push("Inseriu imagem");

// Ctrl+Z desfaz a ÚLTIMA ação (LIFO)
const acao = historico.pop();
console.log("Desfez:", acao); // "Inseriu imagem"

const acao2 = historico.pop();
console.log("Desfez:", acao2); // "Negritou texto"
```

📌 "Desfazer" de editores é uma **pilha**: última ação feita = primeira a ser desfeita → **LIFO**.

📊 Filas — Queue

Uma **fila** é uma estrutura linear que funciona como uma **fila de espera**.

O primeiro a entrar é o primeiro a sair — **FIFO** (*First In, First Out*).



Operações principais

Operação	Descrição
`enqueue()`	Enfileira (insere no **final**)
`dequeue()`	Desenfileira (remove do **início**)
`front()`	Consulta o **primeiro** sem remover

FIFO: o **primeiro** a entrar é o **primeiro** a sair. Entrada pelo **final**, saída pelo **início**.

⚙ Operações de uma fila

```
const fila = [];

// Enqueue — insere no final
fila.push("Ana");        // [Ana]
fila.push("Carlos");     // [Ana, Carlos]

// Dequeue — remove do início
const atendido = fila.shift(); // "Ana"

// Front — consulta o primeiro
console.log(fila[0]); // "Carlos"
```

Estado da fila

Após enqueue("Ana") e enqueue("Carlos"):



Após dequeue() → remove "Ana":



⚠️ `shift()` em array é **O(n)**. Para filas eficientes, use listas com ponteiros.

💻 Fila em JavaScript — exemplo comentado

```
// Fila de impressão de documentos
const filaImpressao = [];

filaImpressao.push("Relatório.pdf");
filaImpressao.push("Contrato.docx");
filaImpressao.push("Apresentação.pptx");

// Primeiro a entrar é o primeiro a sair (FIFO)
const impresso = filaImpressao.shift();
console.log("Imprimiu:", impresso); // "Relatório.pdf"

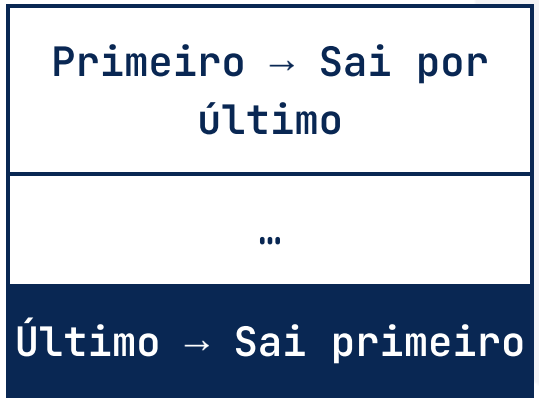
const impresso2 = filaImpressao.shift();
console.log("Imprimiu:", impresso2); // "Contrato.docx"

console.log("Restante:", filaImpressao);
// ["Apresentação.pptx"]
```

📌 Impressão, atendimento, emails — tudo que exige **ordem de chegada** usa **FIFO**.

📊 LIFO × FIFO — Comparação visual

📚 Pilha (LIFO)



- Acesso apenas ao **topo**
- `push()` / `pop()` → **O(1)**
- Exemplos: desfazer, chamada de funções, histórico

🚶 Fila (FIFO)



- Acesso ao **primeiro** e ao **último**
- `enqueue()` / `dequeue()`
- Exemplos: fila de espera, impressão, tarefas

Pilha: **vertical**, último a entrar sai primeiro. Fila: **horizontal**, primeiro a entrar sai primeiro.

⚖️ Comparação entre as estruturas

Característica	Array	Lista Encadeada	Pilha	Fila
Organização	Contígua	Nós ligados	LIFO	FIFO
Acesso por índice	✅ O(1)	❌ O(n)	❌ Só topo	❌ Só frente
Inserir no início	O(n)	**O(1)**	—	—
Inserir no final	**O(1)**	O(n)*	**O(1)**	**O(1)**
Remover do início	O(n)	**O(1)**	—	**O(1)**
Remover do final	**O(1)**	O(n)	**O(1)**	—
Uso de memória	Compacto	+ponteiro por nó	Variável	Variável

💡 * Inserir no final de uma lista simplesmente encadeada é O(n) sem cauda. Com referência à cauda, torna-se O(1).

🌍 Aplicações em Engenharia de Software

FACULDADE FACINT

↩️ Desfazer (Ctrl+Z)
Pilha — histórico de ações

📞 Atendimento
Fila — ordem de chegada

🖨️ Impressão
Fila — documentos em espera

🔙 Voltar (navegador)
Pilha — páginas visitadas

🛒 Carrinho de compras
Array/Lista — itens dinâmicos

⚙️ Chamada de funções
Pilha — call stack do JS

📧 Fila de emails
Fila — envio em ordem

🎮 Buffer de teclas
Fila — entradas sequenciais

Toda aplicação real usa **combinações** de estruturas lineares. A escolha depende do **comportamento** que o sistema exige.

Síntese dos principais conceitos

- **Estrutura linear**: elementos em **sequência**
- **Array**: acesso **O(1)** por índice, inserção no início é **O(n)**
- **Lista encadeada**: inserção/remoção **O(1)** (se na posição), acesso **O(n)**
- **Pilha (LIFO)**: último a entrar, primeiro a sair
- **Fila (FIFO)**: primeiro a entrar, primeiro a sair
- Arrays são **contíguos**, listas usam **ponteiros**
- A escolha da estrutura depende das **operações mais frequentes**
- JavaScript: arrays são **dinâmicos** e flexíveis

🧭 Não existe estrutura "melhor": existe a estrutura **mais adequada ao problema**.

📝 Atividade — Pilha e Fila

```
// PILHA — Analise e responda:
const pilha = [];
pilha.push("A");  pilha.push("B");  pilha.push("C");
const x = pilha.pop();      // x = ?
pilha.push("D");
const y = pilha.pop();      // y = ?
```

```
// FILA — Analise e responda:
const fila = [];
fila.push("1");  fila.push("2");  fila.push("3");
const a = fila.shift();     // a = ?
fila.push("4");
const b = fila.shift();     // b = ?
```

💡 Execute no console para verificar. Pergunte-se: qual o valor de **x, y, a, b**? O que sobrou em cada estrutura?

📝 Atividade — Reflexão

↩️ Desfazer (Ctrl+Z)

Qual estrutura usaria? Justifique com **LIFO** ou **FIFO**.

🏦 Atendimento bancário

Qual estrutura é mais adequada? Por quê?

📦 Inserção no início

Array `[10, 20, 30]` precisa receber `5` no início. Qual operação e custo?

🔗 Lista × Array

Em que situação uma lista encadeada é melhor que um array?

🧠 **Dica:** pense em cada cenário antes de responder. Não existe resposta errada quando há justificativa.

📖 Glossário

Termo	Definição
Estrutura linear	Estrutura de dados onde os elementos são organizados em sequência, um após o outro
Array (Vetor)	Estrutura que armazena elementos em posições contíguas de memória, acessíveis por índice
Índice	Posição numérica de um elemento dentro de um array, iniciando em 0
Lista encadeada	Estrutura formada por nós, onde cada nó guarda um valor e a referência para o próximo
Nó (Node)	Elemento de uma lista encadeada que contém dado + ponteiro para o próximo nó
Head	Referência ao primeiro nó de uma lista encadeada
Pilha (Stack)	Estrutura LIFO: o último elemento inserido é o primeiro a ser removido
Fila (Queue)	Estrutura FIFO: o primeiro elemento inserido é o primeiro a ser removido

📖 Glossário

Termo	Definição
LIFO	*Last In, First Out* — último a entrar, primeiro a sair
FIFO	*First In, First Out* — primeiro a entrar, primeiro a sair
Push	Operação de inserção em uma pilha (adiciona no topo)
Pop	Operação de remoção em uma pilha (remove do topo)
Peek	Consulta ao topo de uma pilha sem remover o elemento
Enqueue	Operação de inserção em uma fila (adiciona no final)
Dequeue	Operação de remoção em uma fila (remove do início)
Big O	Notação que descreve o crescimento do custo de um algoritmo no pior caso